# COMPUTER NETWORKS LABORATORY Faculty Manual

## Department of Computer Science and Engineering

**Course Title:** Computer Networks Laboratory

**Course Code: _____**

**Semester: _____**

**Academic Year: _____**

**Prepared by: _____**

**Approved by: _____**

**Contents**

Vision and Mission

Course Objectives

Course Outcomes

Laboratory Rules and Regulations

Software and Hardware Requirements

Weekly Laboratory Schedule

List of Experiments

Experiment Details

Assessment Rubrics

Viva-Voce Questions

Model Record Format

References

**1. Vision**

To produce competent graduates with strong theoretical and practical knowledge in computer networks capable of designing, implementing, and analyzing communication systems.

**2. Mission**

Provide practical exposure to networking concepts.

Develop programming skills for networking algorithms.

Train students in network analysis tools.

Encourage analytical thinking and problem-solving.

**3. Course Objectives**

Upon completion of this laboratory, students will be able to:

Understand networking hardware and LAN setup.

Implement Data Link Layer protocols.

Develop programs for error detection and correction.

Implement routing algorithms.

Analyze network traffic using Wireshark.

Perform network scanning using Nmap.

Simulate network protocols using NS2.

**4. Course Outcomes (COs)**

| CO | Description |
|---|---|
| CO1 | Understand networking devices and LAN configuration. |
| CO2 | Implement framing and error detection techniques. |
| CO3 | Develop routing protocol algorithms. |
| CO4 | Analyze network performance using software tools. |
| CO5 | Simulate networking protocols and evaluate performance. |

**5. Laboratory Rules**

Students must attend the lab in uniform (if applicable).

Carry the laboratory record.

Complete pre-lab preparation.

Save all programs before leaving.

Follow laboratory safety rules.

Internet usage only for academic purposes.

Mobile phones are not permitted.

**6. Hardware Requirements**

Desktop Computers

LAN Switch

Router

CAT6 Ethernet Cables

RJ45 Connectors

Crimping Tool

**7. Software Requirements**

Ubuntu Linux

GCC Compiler

Wireshark

Nmap

NS2 Simulator

Visual Studio Code / Code::Blocks

| Week | Experiment |
|---|---|
| 1 | Study of Networking Devices and LAN |
| 2 | Character Stuffing |
| 3 | Bit Stuffing |
| 4 | Checksum |
| 5 | Hamming Code |
| 6 | CRC Implementation |
| 7 | Stop and Wait Protocol |
| 8 | Go-Back-N |
| 9 | Selective Repeat |
| 10 | Leaky Bucket Algorithm |
| 11 | Dijkstra Algorithm |
| 12 | Distance Vector Routing |
| 13 | Broadcast Tree |

14 Wireshark Packet Capture

15 Nmap Scanning

16 NS2 Simulations

**Experiment Format**

Each experiment should contain:

Aim

Objectives

Theory

Algorithm

Flowchart

Program

Sample Input

Sample Output

Result

**Experiment 1**

Study of Network Devices and LAN

**Aim**

To study various networking devices and establish communication in a Local Area Network.

Objectives

Study networking hardware.

Connect computers using LAN.

Verify communication.

Equipment Required

PCs

Switch

Ethernet Cable

Router

Theory

**Discuss:**

Hub

Switch

Router

Bridge

Repeater

Gateway

Modem

NIC

Procedure

Connect PCs using switch.

Assign IP addresses.

Verify using ping.

Transfer files.

Expected Result

Successful communication between computers.

**Experiment 2**

Character Stuffing & Bit Stuffing

**Aim**

To implement framing techniques.

Theory

Character Stuffing

Escape Character

Bit Stuffing

Flag Bits

Algorithm

Read input.

Apply stuffing.

Display framed data.

**Program for Character Stuffing**

```
#include<stdio.h>
#include<conio.h>
#include<string.h>
void main(){
    int i=0,j=0,n,pos;
    char a[20],b[50],ch;
    printf("Enter string\n");
    scanf("%s",&a);
    n=strlen(a);
    b[0]='d';
    b[1]='l';
    b[2]='e';
    b[3]='s';
    b[4]='t';
    b[5]='x';
    j=6;
    while(i<n){
        if(a[i]=='d' && a[i+1]=='1' && a[i+2]=='e')
        {
            b[j]='d';
            b[j+1]='1';
            b[j+2]='e';
            j=j+3;
        }
        b[j]=a[i];
        i++;
```

```
        j++;
    }
    b[j]='d';
    b[j+1]='l';
    b[j+2]='e';
    b[j+3]='e';
    b[j+4]='t';
    b[j+5]='x';
    b[j+6]='\0';
    printf("\nframe after stuffing:\n");
    printf("%s",b);
}
```

**Result**

**Input**

Enter string

computer

**Output**

frame after stuffing:

dlestxcomputerdleetx

**Program for Bit Stuffing**

```
#include<stdio.h>
#include<string.h>

void main()
{
    char input[100],stuffed[200];
    int i,j=0,count=0;
    printf("Enter binary data (0s and 1s): ");
    scanf("%s",input);
    for(i=0;input[i]!='\0';i++){
        stuffed[j++]=input[i];
        if(input[i]=='1'){
            count++;
            if(count==5){
                stuffed[j++]= '0';
                count=0;
            }
        }
        else
        count=0;
    }
    stuffed[j]='\0';
    printf("Stuffed Data:%s\n",stuffed);
}
```

**Result**

**Input**

Enter binary data (0s and 1s): 111110

**Output**

Stuffed Data:111

**Experiment 3**

**Checksum**

**Aim**

Implement checksum error detection.

Theory

One's complement addition.

Algorithm

Divide message.

Add segments.

Complement result.

Verify at receiver.

## Program for Checksum:

```
#include<stdio.h>
void main(){
    int data[100],length,i,sum=0,checksum;
    printf("Enter length of data: ");
    scanf("%d",&length);
    printf("Enter %d data integers :\n", length);
    for(i=0;i<length;i++){
        scanf("%d",&data[i]);
        sum+=data[i];
    }
    checksum = ~sum;
    printf("checksum (sender side): %d\n",checksum);
    int receivedSum= 0;
    for(i=0;i<length;i++){
            receivedSum +=data[i];
    }
    receivedSum +=checksum;
    if(~receivedSum==0)
        printf("No error: Data received correctly.\n");
    else
        printf("Error detected in received data.\n");
        }
```

**output:**

Enter length of data: 4

Enter 4 data integers :
20
30
40
50
checksum (sender side): -141
No error: Data received correctly.

**Experiment 4**

**Hamming Code**

**Aim**

Generate Hamming Code.

Theory

Redundant bits

Even parity

Error detection

Error correction

## Hamming Code:

```
#include <stdio.h>
int calcParity(int code[], int pos[], int size) {
    int parity = 0;
    int i;
    for (i = 0; i < size; i++) {
        parity ^= code[pos[i]];
    }
    return parity;
}
int main() {
    int data[4];
    int hammingCode[7];
    int i;
    for (i = 0; i < 7; i++) {
        hammingCode[i] = 0;
    }
    printf("Enter 4 data bits (0 or 1):\n");
    for (i = 0; i < 4; i++) {
        scanf("%d", &data[i]);
    }
    hammingCode[2] = data[0];
    hammingCode[4] = data[1];
    hammingCode[5] = data[2];
    hammingCode[6] = data[3];

    int p1[] = {2, 4, 6};
    int p2[] = {2, 5, 6};
    int p4[] = {4, 5, 6};

    hammingCode[0] = calcParity(hammingCode, p1, 3);
    hammingCode[1] = calcParity(hammingCode, p2, 3);
    hammingCode[3] = calcParity(hammingCode, p4, 3);
```

```
    printf("Generated 7-bit Hamming code: ");
    for (i = 0; i < 7; i++) {
        printf("%d", hammingCode[i]);
    }
    printf("\n");
    return 0;
}
```

**output:**

```
Enter 4 data bits (0 or 1):
1
1
0
1
Generated 7-bit Hamming code: 1010101
```

**Experiment 5**

**CRC**

**Aim**

Implement CRC-12, CRC-16, CRC-CCITT.

Theory

Polynomial division.

## Program for CRC-12:

```
#include <stdio.h>
#include <string.h>

#define CRC_BITS 12


void crc_divide(int bits[], int total_bits, int poly[]) {
    int i, j;
    for (i = 0; i < total_bits - CRC_BITS; i++) {
        if (bits[i] == 1) {
            for (j = 0; j <= CRC_BITS; j++) {
                bits[i + j] ^= poly[j];
            }
        }
    }
}

int main() {
    char data[100];
    int poly[CRC_BITS + 1] = {1,1,0,0,0,0,0,0,0,1,1,1,1};
    int bits[800], i, j, len, total_bits;

    printf("Enter data string: ");
    scanf("%s", data);

    len = strlen(data);
    total_bits = len * 8 + CRC_BITS;


    for (i = 0; i < len; i++) {
        for (j = 0; j < 8; j++) {
            bits[i * 8 + j] = (data[i] >> (7 - j)) & 1;
        }
    }
    for (i = len * 8; i < total_bits; i++) {
        bits[i] = 0;
    }

    int temp[800];
```

```
    for (i = 0; i < total_bits; i++) temp[i] = bits[i];
    crc_divide(temp, total_bits, poly);

    printf("\nCRC-12 bits: ");
    for (i = total_bits - CRC_BITS; i < total_bits; i++) {
        printf("%d", temp[i]);
        bits[i] = temp[i]; // Append CRC to original bits
    }

    printf("\nTransmitted frame: ");
    for (i = 0; i < total_bits; i++) {
        printf("%d", bits[i]);
    }
    printf("\n");


    for (i = 0; i < total_bits; i++) temp[i] = bits[i];
    crc_divide(temp, total_bits, poly);

    int error = 0;
    for (i = total_bits - CRC_BITS; i < total_bits; i++) {
        if (temp[i] != 0) { error = 1; break; }
    }

    if (error == 0) printf("No error detected.\n");
    else printf("Error detected!\n");

    return 0;
}
```

OUTPUT:

```
Enter data string: 1011

CRC-12 bits: 010101011110
Transmitted frame: 001100010011000000110001001100010101011110
No error detected.
```

**CRC-16**

```
#include <stdio.h>
#include <string.h>

#define CRC_BITS 16

void crc_divide(int bits[], int total_bits, int poly[]) {
    int i, j;
    for (i = 0; i < total_bits - CRC_BITS; i++) {
        if (bits[i] == 1) {
            for (j = 0; j <= CRC_BITS; j++) {
                bits[i + j] ^= poly[j];
            }
        }
    }
}

int main() {
    char data[100];
    int poly[CRC_BITS + 1] = {1,1,0,0,0,0,0,0,0,0,0,0,0,0,1,0,1};
    int bits[1000], temp[1000];
    int i, j, len, total_bits;

    printf("Enter data string: ");
    scanf("%s", data);

    len = strlen(data);
    total_bits = len * 8 + CRC_BITS;


    for (i = 0; i < len; i++) {
        for (j = 0; j < 8; j++) {
            bits[i*8 + j] = (data[i] >> (7 - j)) & 1;
        }
    }
    for (i = len * 8; i < total_bits; i++) bits[i] = 0;

    for (i = 0; i < total_bits; i++) temp[i] = bits[i];
    crc_divide(temp, total_bits, poly);

    printf("\nCRC-16 bits: ");
    for (i = total_bits - CRC_BITS; i < total_bits; i++) {
        printf("%d", temp[i]);
        bits[i] = temp[i];
    }

    printf("\nTransmitted frame: ");
    for (i = 0; i < total_bits; i++) printf("%d", bits[i]);
    printf("\n");

    for (i = 0; i < total_bits; i++) temp[i] = bits[i];
```

```
    crc_divide(temp, total_bits, poly);

    int error = 0;
    for (i = total_bits - CRC_BITS; i < total_bits; i++) {
        if (temp[i] != 0) { error = 1; break; }
    }

    if (!error) printf("No error detected.\n");
    else printf("Error detected!\n");

    return 0;
}
```

OUTPUT:

Enter data string: 1011

CRC-16 bits: 1111000101101111
Transmitted frame: 0011000100110000001100010011000111110001011011111
No error detected.

**CRCCCIT:**

```
#include <stdio.h>
#include <string.h>

#define CRC_BITS 16

void crc_divide(int bits[], int total_bits, int poly[]) {
    int i, j;
    for (i = 0; i < total_bits - CRC_BITS; i++) {
        if (bits[i] == 1) {
            for (j = 0; j <= CRC_BITS; j++) {
                bits[i + j] ^= poly[j];
            }
        }
    }
}

int main() {
    char data[100];
    int poly[CRC_BITS + 1] = {1,0,0,0,1,0,0,0,0,0,0,1,0,0,0,0,1}; // CRC-CCITT polynomial
    int bits[1000], temp[1000];
    int i, j, len, total_bits;

    printf("Enter data string: ");
    scanf("%s", data);

    len = strlen(data);
    total_bits = len * 8 + CRC_BITS;


    for (i = 0; i < len; i++) {
        for (j = 0; j < 8; j++) {
            bits[i*8 + j] = (data[i] >> (7 - j)) & 1;
        }
    }
    for (i = len * 8; i < total_bits; i++) bits[i] = 0;

    for (i = 0; i < total_bits; i++) temp[i] = bits[i];
    crc_divide(temp, total_bits, poly);

    printf("\nCRC-CCITT bits: ");
    for (i = total_bits - CRC_BITS; i < total_bits; i++) {
        printf("%d", temp[i]);
        bits[i] = temp[i];
    }

    printf("\nTransmitted frame: ");
    for (i = 0; i < total_bits; i++) printf("%d", bits[i]);
    printf("\n");

    for (i = 0; i < total_bits; i++) temp[i] = bits[i];
    crc_divide(temp, total_bits, poly);
```

```
    int error = 0;
    for (i = total_bits - CRC_BITS; i < total_bits; i++) {
        if (temp[i] != 0) { error = 1; break; }
    }

    if (!error) printf("No error detected.\n");
    else printf("Error detected!\n");

    return 0;
}
```

**OUTPUT:**

Enter data string: 1011

CRC-CCITT bits: 1000111100101110
Transmitted frame: 0011000100110000001100010011000110001111100101110
No error detected.

**Experiment 6**

Stop and Wait Protocol

Theory

Reliable transmission using acknowledgments.

## Program for Stop&Wait Protocol:

```
#include<stdio.h>
#include<stdlib.h>
#include<time.h>

void sender(int totalFrames);
void receiver(int frame);

void main(){
    int totalFrames;
    printf("=== Stop-and-Wait protocal Simulation ===\n");
    printf("Enter total number of frames to send: ");
    scanf("%d",&totalFrames);
    sender(totalFrames);
}
void sender(int totalFrames){
    int i,ack,choice;
    srand(time(NULL));

    for(i=0;i<=totalFrames;i++){
        printf("\nSender: Sending Frame %d", i);
        printf("\nReceiver: Do you want to ACK Frame %d? (1=Yes,0=Lost): ",i);
        scanf("%d",&choice);

        if(choice==1){
            receiver(i);
            printf("Sender: ACK %D received \n",i);
        }
        else
        {
            printf("Sender: ACK %d lost. Retransmitting...\n",i);
            i--;
        }
    }
    printf("\nAll %d frames sent succesfully!\n", totalFrames);
}
void receiver(int frame){
    printf("Receiver:Frame %d received. Sending ACK %D...\n", frame,frame);
}
```

**output:**

=== Stop-and-Wait protocal Simulation ===
Enter total number of frames to send: 5

Sender: Sending Frame 0
Receiver: Do you want to ACK Frame 0? (1=Yes,0=Lost): 1
Receiver:Frame 0 received. Sending ACK D...
Sender: ACK D received

Sender: Sending Frame 1
Receiver: Do you want to ACK Frame 1? (1=Yes,0=Lost): 1
Receiver:Frame 1 received. Sending ACK D...
Sender: ACK D received

Sender: Sending Frame 2
Receiver: Do you want to ACK Frame 2? (1=Yes,0=Lost): 1
Receiver:Frame 2 received. Sending ACK D...
Sender: ACK D received

Sender: Sending Frame 3
Receiver: Do you want to ACK Frame 3? (1=Yes,0=Lost): 0
Sender: ACK 3 lost. Retransmitting...

Sender: Sending Frame 3
Receiver: Do you want to ACK Frame 3? (1=Yes,0=Lost): 1
Receiver:Frame 3 received. Sending ACK D...
Sender: ACK D received

Sender: Sending Frame 4
Receiver: Do you want to ACK Frame 4? (1=Yes,0=Lost): 1
Receiver:Frame 4 received. Sending ACK D...
Sender: ACK D received

Sender: Sending Frame 5
Receiver: Do you want to ACK Frame 5? (1=Yes,0=Lost): 1
Receiver:Frame 5 received. Sending ACK D...
Sender: ACK D received

All 5 frames sent succesfully!

**Experiment 7**

Go-Back-N

Theory

Sliding window with retransmission.

```
#include<stdio.h>
void main()
{
   int total_frames,window_size;
   int sent=0,lost_frame,i;
   printf("Enter total number of frames to send: ");
   scanf("%d",&total_frames);
   printf("Enter window size: ");
   scanf("%d",&window_size);
   printf("\n--- Go-Back-N Sliding Window Protocol Simulation ---\n\n");
   while(sent<total_frames)
   {
     printf("Sender: sending frames
              ");
     for(i=sent;i<sent+window_size && i<total_frames;i++)
         printf("%d ",i);
     printf("\n");
     printf("Enter the frame number to be lost (or -1 if none lost): ");
     scanf("%d",&lost_frame);
     if(lost_frame >= sent && lost_frame < sent+window_size && lost_frame < total_frames)
     {
          printf("Receiver: Frame %d lost! Go back and resend from %d\n\n",lost_frame,lost_frame);
          sent=lost_frame;
     }
     else
     {
          int ack=sent+window_size;
          if(ack>total_frames)
              ack=total_frames;
          printf("Receiver: Acknowledged up to frame %d\n\n",ack-1);
          sent=ack;
     }
   }
  printf("All frames sent successfully using Go-Back-N!\n");
}
```

**Output:**

```
Enter total number of frames to send: 7
Enter window size: 3

--- Go-Back-N Sliding Window Protocol Simulation ---

Sender: sending frames0 1 2
Enter the frame number to be lost (or -1 if none lost): 4
```

Receiver: Acknowledged up to frame 2

Sender: sending frames3 4 5
Enter the frame number to be lost (or -1 if none lost): 3
Receiver: Frame 3 lost! Go back and resend from 3

Sender: sending frames3 4 5
Enter the frame number to be lost (or -1 if none lost): 5
Receiver: Frame 5 lost! Go back and resend from 5

Sender: sending frames5 6
Enter the frame number to be lost (or -1 if none lost): 1
Receiver: Acknowledged up to frame 6

All frames sent successfully using Go-Back-N!

Process returned 0 (0x0)  execution time : 53.457 s
Press any key to continue.

**Experiment 8**

Selective Repeat

Theory

Selective retransmission of erroneous frames.

**Program for Selective Repeat**

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 50
void main(){
 int totalFrames, windowSize;
 int frameStatus[MAX];
 int i, choice, resend;
printf("\n=== Selective Repeat ARQ Simulation ==\n");
printf("Enter total number of frames to send: ");
scanf("%d", &totalFrames);
printf("Enter window size: ");
scanf("%d", &windowSize);
for(i=0;i<totalFrames;i++)
 frameStatus[i]=0;
int sent = 0;
while(sent < totalFrames){
printf("\n--- Sending Window of Frames ---\n");
for(i=sent;i<sent+windowSize && i< totalFrames; i++){
  if(frameStatus[i] == 0)
      printf("Frame %d sent.\n", i);
}
for(i=sent;i<sent+windowSize && i< totalFrames; i++){
     if(frameStatus[i] == 0){
          printf("Did frame %d get ACK? (1=yes, 0=Lost): ",i);
          scanf("%d", &choice);
          if (choice ==1){
               printf("ACK for frame %d received.\n",i);
               frameStatus[i] = 1;
```

```
        }
        else
            printf("Frame %d lost / not acknowledged.\n",i);

    }
}
printf("\nDo you want to resend any lost frame? (Enter frame number, -1 to stop): ");
scanf("%d", &resend);
while(resend != -1)
{
    if (resend >= sent && resend < totalFrames && frameStatus[resend]==0)
    {
        printf("Resending frame %d...\n, resend");
        printf("ACK for frame %d received now.\n",resend );
        frameStatus[resend ] = 1;
    }
    else
        printf("Invalid   frame number or already ACKed.\n ");
    printf("Enter next frame to resend (-1 to stop):");
    scanf("%d",&resend);
}
while(sent < totalFrames   && frameStatus[sent]==1)
    sent ++;
}
printf("\n All frames sent and acknowledged sucessfully!\n");
}
```

**Output**

**=== Selective Repeat ARQ Simulation ===**

**Enter total number of frames to send: 4**

**Enter window size: 2**

**--- Sending Window of Frames ---**

**Frame 0 sent.**

**Frame 1 sent.**

**Did frame 0 get ACK? (1=yes, 0=Lost): 1**

**ACK for frame 0 received.**

**Did frame 1 get ACK? (1=yes, 0=Lost): 1**

**ACK for frame 1 received.**

**Do you want to resend any lost frame? (Enter frame number, -1 to stop): -1**

**--- Sending Window of Frames ---**

**Frame 2 sent.**

**Frame 3 sent.**

**Did frame 2 get ACK? (1=yes, 0=Lost): 1**

**ACK for frame 2 received.**

**Did frame 3 get ACK? (1=yes, 0=Lost): 1**

**ACK for frame 3 received.**

**Do you want to resend any lost frame? (Enter frame number, -1 to stop): -1**

**All frames sent and acknowledged successfully!**

**Experiment 9**

Leaky Bucket

Theory

Traffic shaping.

## Program for Leaky Bucket:

```
#include<stdio.h>
#include<stdlib.h>

void main(){
    int bucketSize,outputRate,n,incoming;
    int stored=0;
    printf("=== Leaky Bucket Algorithm Simulation ===\n");

    printf("Enter bucket capacity(in packets): ");
    scanf("%d",&bucketSize);

     printf("Enter output rate (packets per second): ");
    scanf("%d",&outputRate);

    printf("Enter number of incoming packets (time slots): ");
    scanf("%d",&n);
    for(int i=1;i<=n;i++){
        printf("\nTime %d: Enter number of incoming packets: ",i);
        scanf("%d",&incoming);
        printf("Incoming packets: %d\n",incoming);
        if(incoming + stored>bucketSize)
        {
            int dropped = (incoming + stored)-bucketSize;
            stored = bucketSize;
            printf("Bucket overflow! Dropped packets: %d\n", dropped);
        }
        else
            stored = stored+incoming;
        if (stored > outputRate)
        {
            stored=stored-outputRate;
            printf("Transmitted: %d | Packets left in bucket:%d\n",outputRate, stored);
        }
        else
        {

            printf("Transmitted: %d | packets left in bucket: 0\n", stored);
            stored=0;
        }
    }
    while(stored>0){
        if(stored>outputRate){
            stored=stored-outputRate;
            printf("\nTransmitted:%d | Packets left in bucket:%d\n", outputRate, stored);
```

```
        }
        else{
            printf("Transmitted: %d | Packets left in bucket: 0\n", stored);
            stored=0;
        }
    }
    printf("\nAll packets transmitted successfully!\n");

}
```

**output:**

```
=== Leaky Bucket Algorithm Simulation ===Enter bucket capacity(in packets): 10
Enter output rate (packets per second): 5
Enter number of incoming packets (time slots): 4

Time 1: Enter number of incoming packets: 6
Incoming packets: 6
Transmitted: 5 | Packets left in bucket:1

Time 2: Enter number of incoming packets: 8
Incoming packets: 8
Transmitted: 5 | Packets left in bucket:4

Time 3: Enter number of incoming packets: 9
Incoming packets: 9
Bucket overflow! Dropped packets: 3
Transmitted: 5 | Packets left in bucket:5

Time 4: Enter number of incoming packets: 9
Incoming packets: 9
Bucket overflow! Dropped packets: 4
Transmitted: 5 | Packets left in bucket:5
Transmitted: 5 | Packets left in bucket: 0

All packets transmitted successfully!
```

**Experiment 10**

Dijkstra Algorithm

Theory

Shortest path computation.

## Dijkistra's Algorithm:

```
#include<stdio.H>
#include<limits.h>
#include<stdbool.h>

int minDistance(int dist[], bool visited[],int V){
    int min = INT_MAX, min_index = -1;
    for(int i = 0; i < V; i++){
        if(!visited[i] && dist[i] <= min){
        min = dist[i];
        min_index = i;
        }
    }
    return min_index;
}
void printPath(int parent[], int j){
    if(parent[j] == -1){
            printf("%d ", j);
            return;
    }
    printPath(parent, parent[j]);
    printf("-> %d ", j);
}
void dijkstra(int graph[20][20], int V, int src){
    int dist[20];
    bool visited[20];
    int parent[20];
    for(int i=0;i<V;i++){
        dist[i] = INT_MAX;
        visited[i]=false;
        parent[i]=-1;
    }
    dist[src] = 0;
    for(int count = 0; count<V-1;count++){
        int u = minDistance(dist, visited, V);
        visited[u] = true;
        for(int v=0; v<V;v++){
            if(!visited[v] && graph[u][v] && dist[u] != INT_MAX && dist[u] + graph[u][v] < dist[v]){
                dist[v] = dist[u] + graph[u][v];
                parent[v] = u;
            }
        }
    }
    printf("\nVertex\tDistance|tPath|n");
    for(int i=0;i<V;i++){
```

```
        if(dist[i] == INT_MAX)
            printf("%d\tINF\t\tNo path\n", i);
            else
                {
                printf("%d\t%d\t\t", i, dist[i]);
                printPath(parent, i);
                printf("\n");
            }
        }
    }
    void main(){
        int V, src;
        int graph[20][20];
        printf("Enter number of vertices (max 20): ");
        scanf("%d", &V);
        printf("\nEnter the adjacency matrix (use 0 for no edge:\n");
        for(int i =0;i<V;i++){
            for(int j=0;j<V;j++)
                scanf("%d",&graph[i][j]);
        }
        printf("\nEnter the source vertex (0 to %d): ", V-1);
        scanf("%d", &src);
        dijkstra(graph, V, src);
    }
```

**output:**

Enter number of vertices (max 20): 5

Enter the adjacency matrix (use 0 for no edge:
0
5
0
4
9
5
0
4
0
0
0
4
0
2
3
4
0
2
0
2
9
0
3

2
0

Enter the source vertex (0 to 4): 0

Vertex  Distance|tPath|n0  0  0
1  5  0 -> 1
2  6  0 -> 3 -> 2
3  4  0 -> 3
4  6  0 -> 3 -> 4

**Experiment 11**

Distance Vector Routing

Theory

Bellman-Ford routing.

```
#include<stdio.h>
#define INF 999
#define MAX 20

struct RoutingTable
{
    int dist[MAX];
    int nextHop[MAX];
};
void main()
{
   int n,i,j,k;
   int cost[MAX][MAX];
   char nodes[MAX][20];
   struct RoutingTable rt[MAX];

   printf("Enter number of   nodes   (hosts/routers): ");
   scanf("%d",&n);

   printf("Enter node names (e.g., H1,H2, ...):\n");
   for(i=0;i<n;i++)
   {
       printf("Node %d: ",i+1);
       scanf("%s",nodes[i]);
   }

   printf("\nEnter adjacency matrix (delay cost,INF=999,0 for self):\n");
   for(i=0;i<n;i++)
   {
       for(j=0;j<n;j++)
         scanf("%d",&cost[i][j]);
   }
   for(i=0;i<n;i++)
   {
       for(j=0;j<n;j++)
       {
           rt[i].dist[j]=cost[i][j];
           if(cost[i][j]!=INF && i!=j)
             rt[i].nextHop[j]=j;
           else
             rt[i].nextHop[j]=-1;
       }
   }
   int updated;
   do
```

```
    {
        updated=0;
        for(i=0;i<n;i++)
        {
            for(j=0;j<n;j++)
            {
                for(k=0;k<n;k++)
                {
                    if(rt[i].dist[j]>cost[i][k]+rt[k].dist[j])
                    {
                        rt[i].dist[j]=cost[i][k]+rt[k].dist[j];
                        rt[i].nextHop[j]=k;
                        updated =1;
                    }
                }
            }
        }
    }while(updated);
    for(i=0;i<n;i++)
    {
        printf("\nRouting Table for Node %s:\n",nodes[i]);
        printf("Destination\tNextHop\tCost\n");
        for(j=0;j<n;j++)
        {
            if(i==j)continue;
            printf("%s\t\t",nodes[j]);
            if(rt[i].nextHop[j]!=-1)
              printf("%s\t\t",nodes[rt[i].nextHop[j]]);
            else
              printf("-\t\t");
            if(rt[i].dist[j]==INF)
              printf("INF\n");
            else
              printf("%d\n",rt[i].dist[j]);

        }
    }
}
```

output:

```
Enter number of   nodes   (hosts/routers): 4
Enter node names (e.g., H1,H2, ...):
Node 1: h1
Node 2: h2
Node 3: h3
Node 4: h4

Enter adjacency matrix (delay cost,INF=999,0 for self):
0 5 1 999
```

5 0 3 2
1 3 0 2
999 2 2 0

Routing Table for Node h1:

| Destination | NextHop | Cost |
|---|---|---|
| h2 | h3 | 4 |
| h3 | h3 | 1 |
| h4 | h3 | 3 |

Routing Table for Node h2:

| Destination | NextHop | Cost |
|---|---|---|
| h1 | h3 | 4 |
| h3 | h3 | 3 |
| h4 | h4 | 2 |

Routing Table for Node h3:

| Destination | NextHop | Cost |
|---|---|---|
| h1 | h1 | 1 |
| h2 | h2 | 3 |
| h4 | h4 | 2 |

Routing Table for Node h4:

| Destination | NextHop | Cost |
|---|---|---|
| h1 | h3 | 3 |
| h2 | h2 | 2 |
| h3 | h3 | 2 |

**Experiment 12**

Broadcast Tree

Theory

Efficient packet broadcasting using spanning trees.

Experiment 13

Wireshark

Exercises

Packet Capture

Apply Filters

Analyze Protocols

Statistics

Experiment 14

Nmap

Exercises

Host Discovery

Port Scan

Service Detection

OS Detection

Experiment 15

NS2 Simulator

Experiments

Packet Drop

TCP Packet Drop

UDP Packet Drop

Congestion

Throughput Comparison